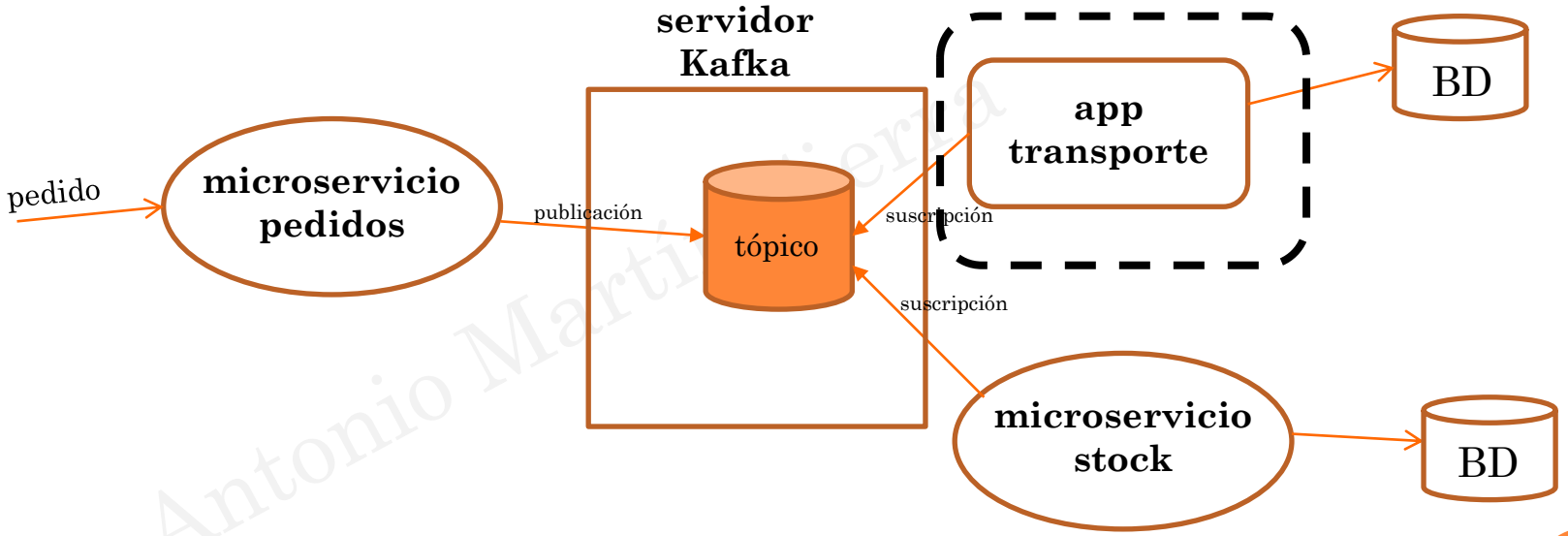
Microservicios y Kafka

Escenario

- **Utilizado para notificar sucesos desde un microservicio a otros microservicios o aplicaciones**



Antonio Martínez Sierra

Microservicio productor

- **Además del starter Web, deberá incluir la dependencia al starter Spring Apache Kafka.**

- **El envío de mensajes al tópico se realizará mediante el objeto KafkaTemplate:**

```
@Autowired
private KafkaTemplate<String, Pedido> kafkaTemplate;
:
CompletableFuture<SendResult<String, Pedido>> future = kafkaTemplate.send(topic, pedido);
```

- **Se deberán configurar una serie de propiedades en el archivo application.properties:**

```
spring.kafka.bootstrap-servers=localhost:9092
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-serializer=org.springframework.kafka.support.serializer.JsonSerializer
topic=pedidosTopic
```

Antonio Martín Sierra

Microservicio consumidor

- **Incluirá también la dependencia al starter Spring Apache Kafka.**

- **Para suscribirse a un tópico, se anotará con @KafkaListener el método donde se recibirán los mensajes:**

```
@KafkaListener(topics = "pedidosTopic", groupId = "myGroup")
 public void listenTopic1(Pedido pedido) {
  :
 }
```

- **Se deberán configurar las siguientes propiedades en el archivo application.properties:**

```
spring.kafka.bootstrap-servers=localhost:9092
spring.kafka.consumer.group-id=myGroup
spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.JsonDeserializer
spring.kafka.consumer.properties.spring.json.trusted.packages=*
```

Antonio Martín Sierra